Adding C Library wrappers for all syscalls

Linux Plumbers Conference 2025

Carlos O'Donell

Distinguished Engineer

Red Hat

What problem are we trying to solve?

- Make it easy to use kernel functionality exposed via syscalls

How do we do that today?

- Provide libraries that encapsulate groupings of functionality
  - liburing, libibverbs, liburcu, libaio, ... libc
- Use the `syscall()` function provided by libc
- Assembly for custom per-arch per-syscall calling conventions



Example “syscall”: userspace-rcu/include/urcu/futex.h:

```
64 static inline int futex(int32_t *uaddr, int op, int32_t val,
65                 const struct timespec *timeout, int32_t *uaddr2, int32_t val3)
66 {
67         return syscall(__NR_futex, uaddr, op, val, timeout,
68                        uaddr2, val3);
69 }
70
```

No `futex()` wrapper.



Example “syscall”: systemd/src/basic/ioprio.h:

```
48 static inline int ioprio_set(int which, int who, int ioprio) {
49         return syscall(__NR_ioprio_set, which, who, ioprio);
50 }
51
52 static inline int ioprio_get(int which, int who) {
53         return syscall(__NR_ioprio_get, which, who);
54 }
```

No `ioprio_set()` or `ioprio_get()` wrappers.



Example “assembly”: go/src/internal/runtime/syscall/linux/asm_linux_amd64.s:

```
25 // Note that this differs from "standard" ABI convention, which would pass 4th
26 // arg in CX, not R10.
27 TEXT ·Syscall6<ABIInternal>(SB),NOSPLIT,$0
28         // a6 already in R9.
29         // a5 already in R8.
30         MOVQ    SI, R10 // a4
31         MOVQ    DI, DX  // a3
32         MOVQ    CX, SI  // a2
33         MOVQ    BX, DI  // a1
34         // num already in AX.
35         SYSCALL
36         CMPQ    AX, $0xfffffffffffff001
37         JLS     ok
38         NEGQ    AX
39         MOVQ    AX, CX  // errno
40         MOVQ    $-1, AX // r1
41         MOVQ    $0, BX  // r2
42         RET
43 ok:
44         // r1 already in AX.
45         MOVQ    DX, BX // r2
46         MOVQ    $0, CX // errno
```

A `syscall()` wrapper exists but would require cgo.



Can we do better?

- Writing userspace applications that use new syscalls would be simpler if they all had wrappers
  - Example: `sched_setattr()`, `sched_getattr()`
    - 2013: New scheduler syscalls added
    - 2016: util-linux `chrt` uses `SCHED_DEADLINE`
    - 2018: systemd CPUSchedulingPolicy RFE (Issue 10034)
    - 2024: glibc: Linux: Add the sched_setattr and sched_getattr functions
- What does ***better*** look like?



Options:

Create a stand-alone library that has only wrappers?

- Pro: Life-cycle can be coupled to the kernel.
  - Add new syscalls as they appear.
- Con: Another library dependency needs to be installed.
- Con: Duplicates complicated ABI logic in multiple places.

Extend existing C libraries?

- Pro: Available in production.
- Pro: Has existing ABI logic and infrastructure
- Pro: Coordinates with existing functionality
- Con: Not a minimal solution





If glibc knows about all the kernel syscalls...

... can we create wrappers for all syscalls?



What should we agree to?

- 2014: glibc's "WIP: Kernel syscall wrappers"
  - 2026: Move it out of WIP.
- Agree to an "Always make a syscall wrapper" even for the footguns?
  - e.g. futex, io_uring, ...
- If a kernel syscall is later specified by ISO C or POSIX?
  - Version the symbol?
- Make sure we don't break existing libraries?
  - liburing should not break if we add io_uring?
- Is it useful even if a higher level library eventually shows up?
  - Yes. But I'm opinionated.



Example: Don't break liburing:

```
carlos@fedora:~/src/liburing$ grep -r 'syscall(' *
examples/helpers.c:     return (int)syscall(SYS_memfd_create, name, flags);
man/io_uring.7:     ret = syscall(__NR_io_uring_setup, entries, p);
man/io_uring.7:     ret = syscall(__NR_io_uring_enter, ring_fd, to_submit,
src/arch/generic/syscall.h:     ret = syscall(__NR_io_uring_register, fd, opcode, arg, nr_args);
src/arch/generic/syscall.h:     ret = syscall(__NR_io_uring_setup, entries, p);
src/arch/generic/syscall.h:     ret = syscall(__NR_io_uring_enter, fd, to_submit, min_complete, flags,
test/35fa71a030ca.c:  syscall(SYS_futex, &ev->state, FUTEX_WAKE | FUTEX_PRIVATE_FLAG);
test/35fa71a030ca.c:    syscall(SYS_futex, &ev->state, FUTEX_WAIT | FUTEX_PRIVATE_FLAG, 0, 0);
test/35fa71a030ca.c:    syscall(SYS_futex, &ev->state, FUTEX_WAIT | FUTEX_PRIVATE_FLAG, 0, &ts);
test/regbuf-merge.c:    uint32_t fd_io_uring = syscall(__NR_io_uring_setup, entries, setup_params);
test/regbuf-merge.c:    syscall(__NR_io_uring_register, r[0], 0ul, 0x20002840ul, 2ul);
test/statx.c:   return syscall(__NR_statx, dfd, path, flags, mask, statxbuf);
carlos@fedora:~/src/liburing$
```

glibc has `memfd_create()`, and `statx()`.
No wrappers for `io_uring_setup()`, `io_uring_enter()`, `io_uring_register()`, `futex()`.



What should we agree to?

- ...
- glibc 2.41: Added sched_getattr(), sched_setattr()
- glibc 2.43: Adding openat2() to the 2026-01 release
  - [v8] linux: Add openat2 (BZ 31664)
- *glibc 2.44: Added the syscall you want added by working with me.*

Agree to review, coach, and mentor syscall wrappers through the process.



Thank you

Red Hat is the world's leading provider of enterprise open source software solutions. Award-winning support, training, and consulting services make Red Hat a trusted adviser to the Fortune 500.


linkedin.com/company/red-hat


youtube.com/@redhat


facebook.com/RedHat


x.com/RedHat

Red Hat